

EVERLEAF

Online Portfolio Template Generator

PROJECT DOCUMENTATION

Prepared by :

GROUP 7:

Nikenji Louis Peñas

Anthony Pedroza

Jayward Perater

Jay boy Gabunada

Jaymark Pahiculay

Course and section: BSIT 2ND YEAR SECTION -2

Instructor:

Mancilla, Jeorge Rey

Alangcas, Arjay

Quibol, Hazel Queen

Submission date: October 9, 2026

Contents

B. Project Description 3

C. Project Objectives 3

D. Technologies Used 3

E. System Pages and Features 4

F. Three Portfolio Templates 7

Simple (stored key: minimal) 7

Modern 9

Creative 10

G. Online Database and Database Structure 11

H. System Flow 11

I. Deployment Information 12

J. Testing and Results 13

K. Limitations and Recommendations 14

Verified limitations and open checks 14

L. Conclusion 14

Appendix: screenshots 14

B. Project Description

EverLeaf is an online portfolio generator intended for students, early-career professionals, and other people who want a

simple way to present their profile and work. The application brings identity, biography, education, experience, skills,

projects, and social links together in a saved portfolio. An owner can preview one of three designs and choose whether to

publish a public portfolio page.

The interface uses a forest-inspired visual identity. The application code is a Next.js project in next-app/; Laravel files at

the repository root are retained as an earlier implementation. The root README identifies the Next.js app as the current

Vercel application.

Evidence boundary: this report describes source code, SQL migration files, the existing screenshot assets, and a successful

local production build. It does not claim that the live Supabase project, Vercel settings, or user workflows were

independently verified.

C. Project Objectives

Objective	Project intent
Portfolio information management	Collect and maintain profile, education, work experience, skills, projects, and social links.
Online persistence	Store portfolio records in Supabase PostgreSQL and image files in Supabase Storage.
Template selection	Offer Simple, Modern, and Creative views that render the same saved portfolio data.
Portfolio generation	Render a preview and a public page for an active portfolio when its owner publishes it.
Deployment	Serve the Next.js application from the provided Vercel URL and connect it to Supabase using environment configuration.

These are documented objectives and code paths. Their end-to-end completion remains subject to the tests in Section J.

D. Technologies Used

Technology	Verified role in this repository
Next.js 16.3.8 and Server Actions	The active app framework, App Router, and server-side portfolio actions.
React and React DOM 19.2.8	UI components and browser rendering.
TypeScript 5.9.3	Typed application source and type checking.
Tailwind CSS 4.3.3; HTML, CSS, and JavaScript	Active styling and browser-facing markup/language technologies.
Supabase Auth	Authentication integration for the active Next.js application.
Supabase Storage	Active portfolio profile and project image storage integration.
Supabase PostgreSQL	Database platform used by the active Next.js application; remote configuration is not verified here.
MySQL (Legacy/local option)	MySQL is available in the Laravel config; project-reported local use with XAMPP is not established by repo configuration. Laravel's example defaults to SQLite.
Laravel 12.69.2 and PHP ^8.2 (Legacy)	Earlier Laravel/Blade application and its PHP requirement; not the active Next.js backend.
Node.js and npm	Next.js development/build tools. No exact Node.js or npm version is pinned; Next.js 16 requires Node.js >=20.9.0.
Composer	Dependency manager for the legacy Laravel/PHP application.
XAMPP	Project-reported local-development tooling; its use is not confirmed by checked-in configuration.
Visual Studio Code	Project-reported source-code editor.
Git and GitHub	Version control and source-code hosting.
Vercel	Application hosting platform associated with the published Next.js site.

Backend architecture: no separate API server is declared in the Next.js package. Next.js server actions and route handlers

mediate authentication and portfolio operations, while Supabase supplies managed authentication, PostgreSQL, and object storage.

E. System Pages and Features

Route or section	What the source implements
/	Welcome page with a Start Your Journey link to the home page.
/home	Landing page with navigation, hero, How It Works feature band, three template cards, About section, and footer. Features is a home-page section, not a separate route.
/login, /signup	Email/password sign-in and account creation; optional template choice is carried in the URL.
/forgot-password, /reset-password, /auth/callback	Password recovery, new password submission, and email code/session callback.
/goodbye, /contact	Post-sign-out confirmation page and contact page route.
/dashboard	Authenticated portfolio list, visibility controls, preview/edit/design links, recovery mode, and delete/restore actions.
/dashboard/new	Portfolio information form and image upload flow.
/dashboard/[id]/edit	Owner edit form for a saved, active portfolio.
/dashboard/[id]/templates	Owner template chooser with Preview and Use design controls.
/dashboard/[id]/preview/[template]	Authenticated owner preview with preview toolbar.
/template-demo/[template]	Public sample preview with sample information.
/p/[slug]	Public portfolio render; query restricts to published, non-deleted records.

The route list was cross-checked against the successful Next.js production build output. This confirms route compilation, not live behavior.

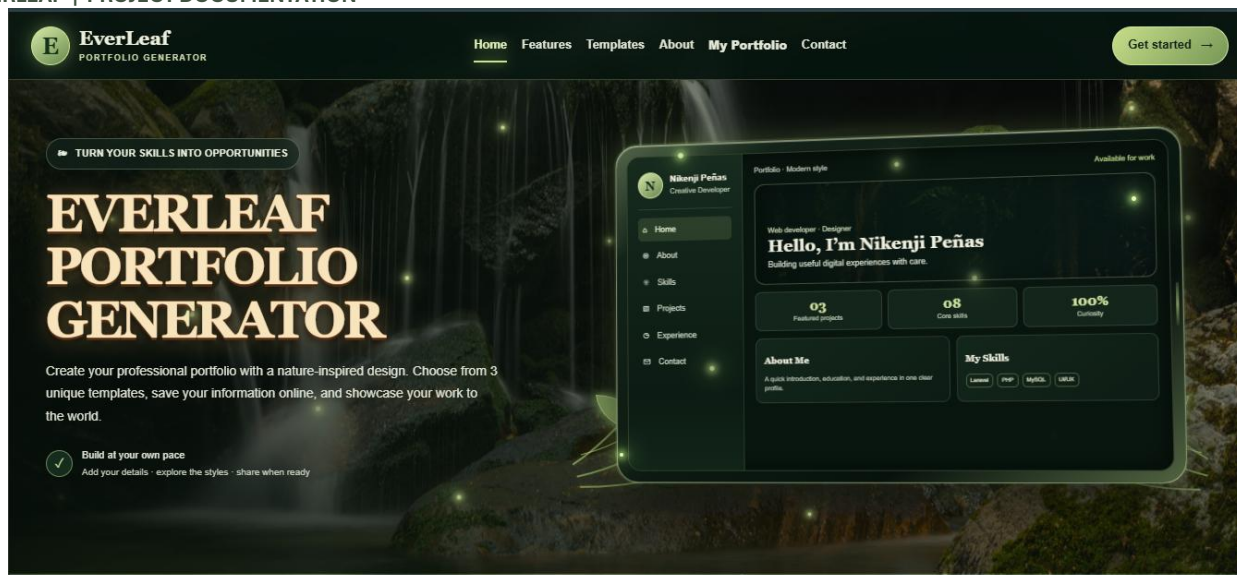


Figure 1. Existing repository desktop screenshot of the EverLeaf landing page.

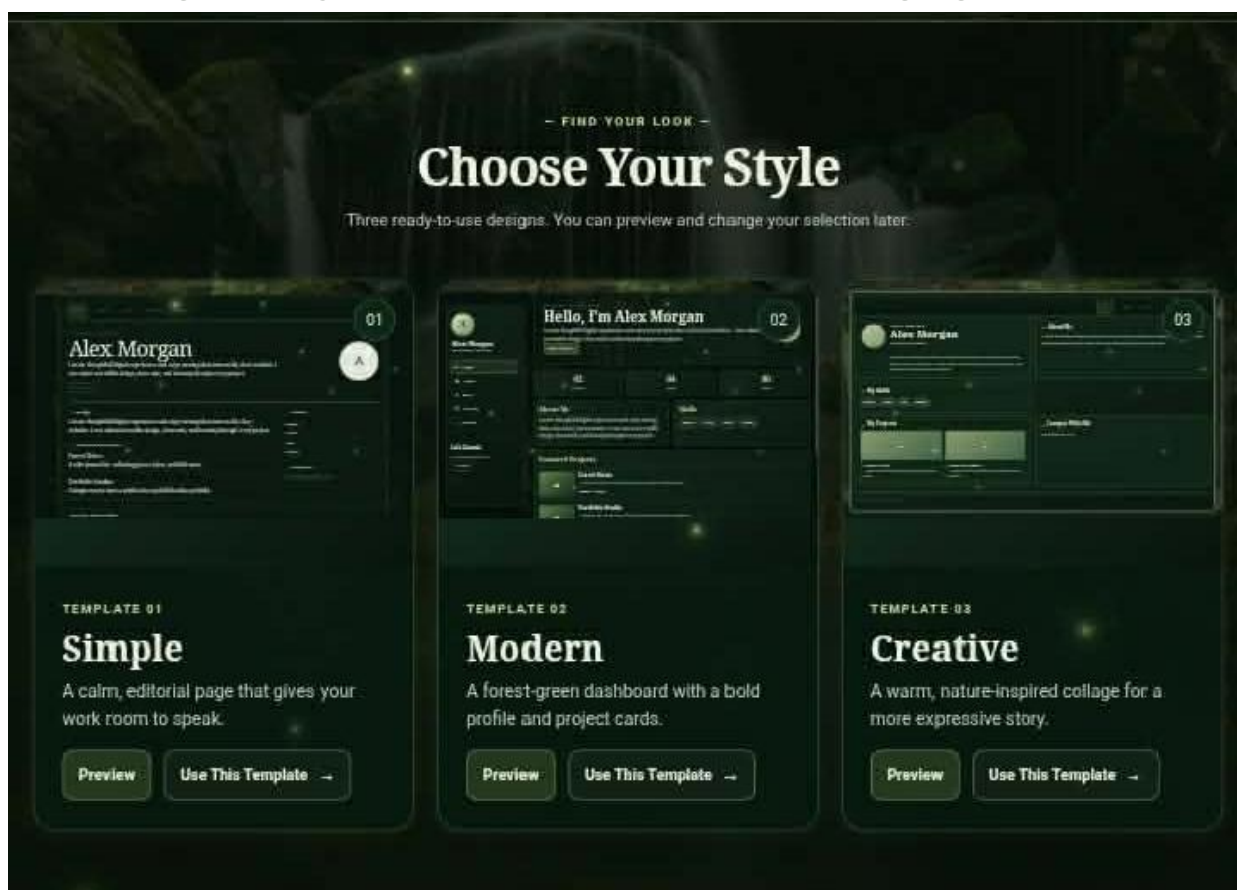


Figure 2. Crop of the landing page How It Works feature band; not a separate Features route.



Figure 3. Existing template-card section from the landing-page screenshot; Preview and Use This Template controls are visible.



BACK TO MY PORTFOLIOS

STEP 1 · YOUR STORY

CREATE PORTFOLIO

Start with the essentials. Add optional details now or later.

Full Name *

Professional Role (Optional)

Software Engineer

Email Address *

Phone Number (Optional)

Address (Optional)

Design

Modern · Forest glass

About Me (Optional)

A short introduction about your work and what you enjoy building.

Skills (Optional) · Separate with commas

TypeScript, React, Supabase

Projects (Optional) · One per line: Title | Description

Portfolio dashboard | A dashboard for creating and sharing portfolios
Community garden map | A map for local growing spaces

Education (Optional)

School

Degree

Bachelor's degree

Field of study

Information Technology

Start date

mm/dd/yyyy

End date

mm/dd/yyyy

Description

☐ Currently studying

REMOVE

+ Add education

Work Experience (Optional)

Position

Company

Figure 4. Existing portfolio form screenshot, upper section. A lower-section image is in the screenshot appendix folder.

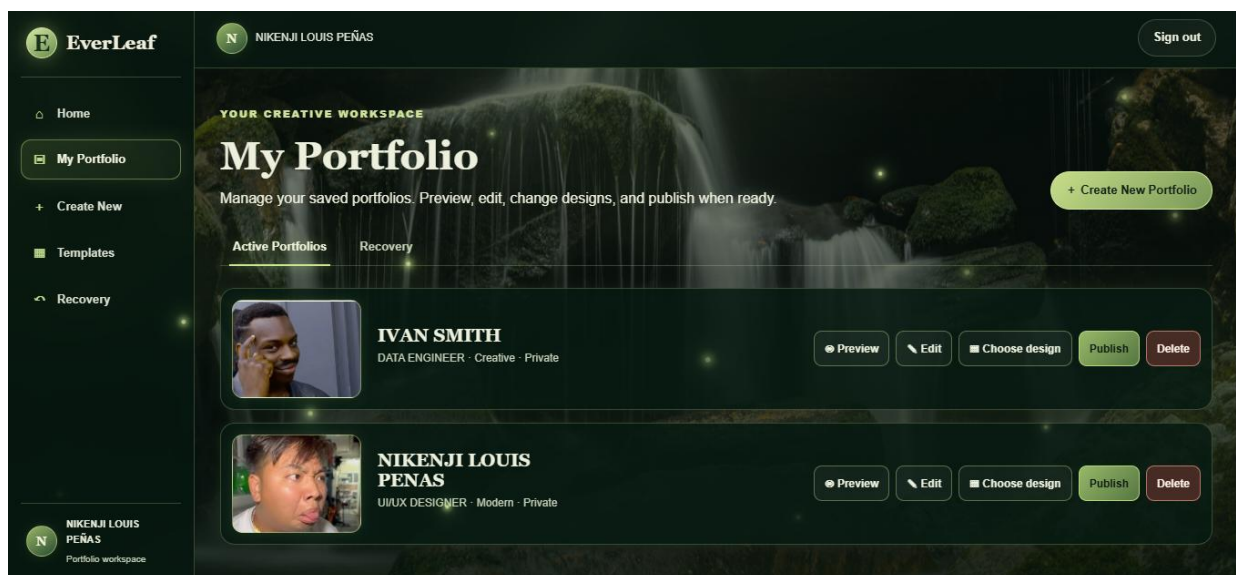


Figure 5. Existing portfolio management screenshot.

The separate portfolio preview capture is saved as 08-portfolio-preview.png. The template-specific preview screenshots in Section F also show the preview toolbar.

F. Three Portfolio Templates

The stored template keys are minimal, modern, and creative. The user-facing label Simple maps to minimal. The

Next.js renderer selects three separate React components: MinimalTemplate, ModernTemplate, and CreativeTemplate.

Each component receives the saved portfolio data, resolved profile/project image URLs, and selected template key. The

descriptions below reflect the component names, chooser copy, styles, and existing screenshots; interactive differences

were not manually tested.

Simple (stored key: minimal)

An editorial layout with restrained spacing and clear profile sections. The source component displays identity, about,

education, experience, skills, selected projects, and contact information where data exists.

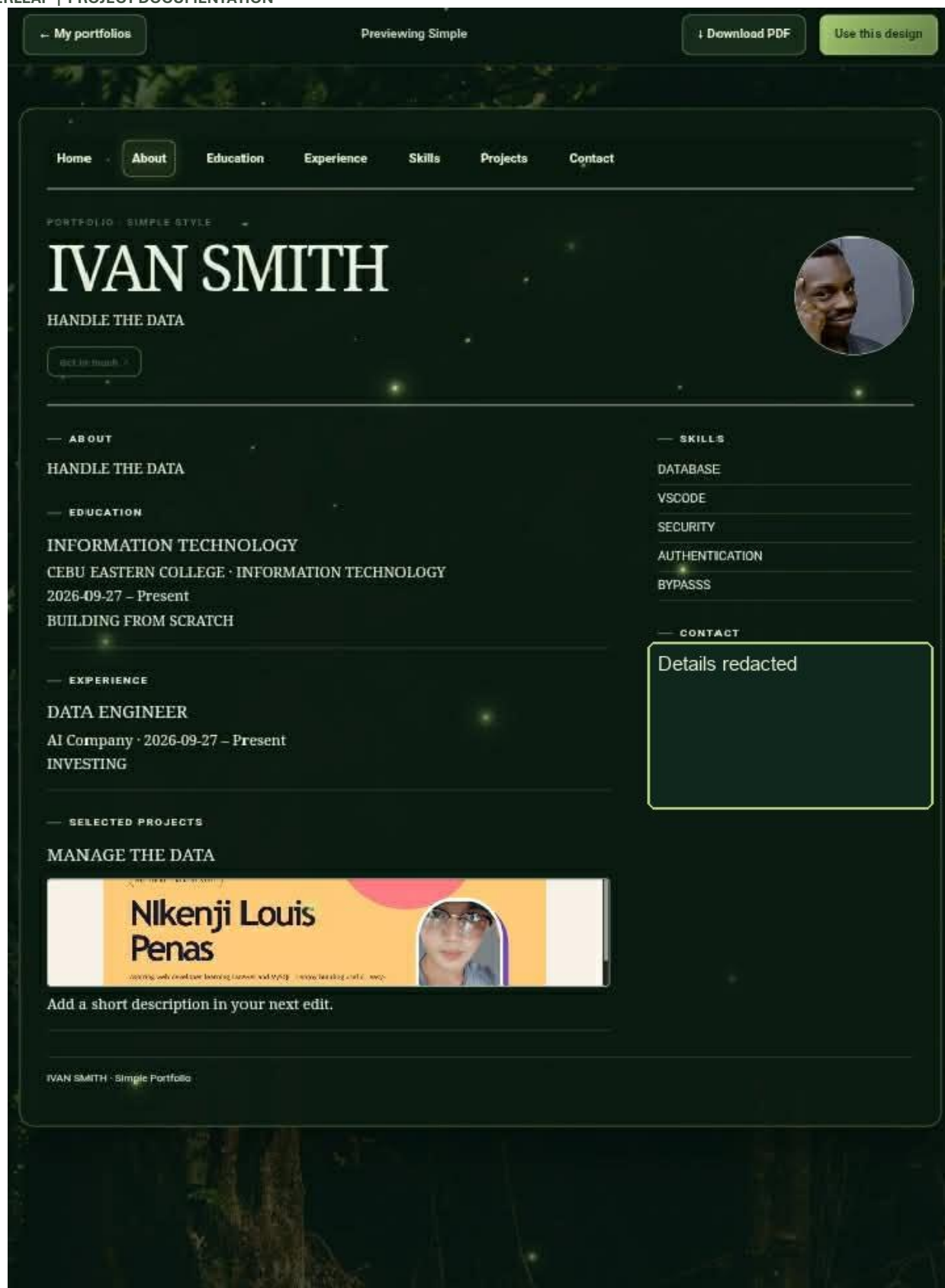


Figure 6. Existing Simple preview screenshot. Personal contact values are redacted in this documentation copy.

Modern

A forest-green dashboard composition with a prominent introduction, sidebar navigation, summary cards, and modular profile/project panels. It is a separate component and stylesheet, not a generic variant of Simple.

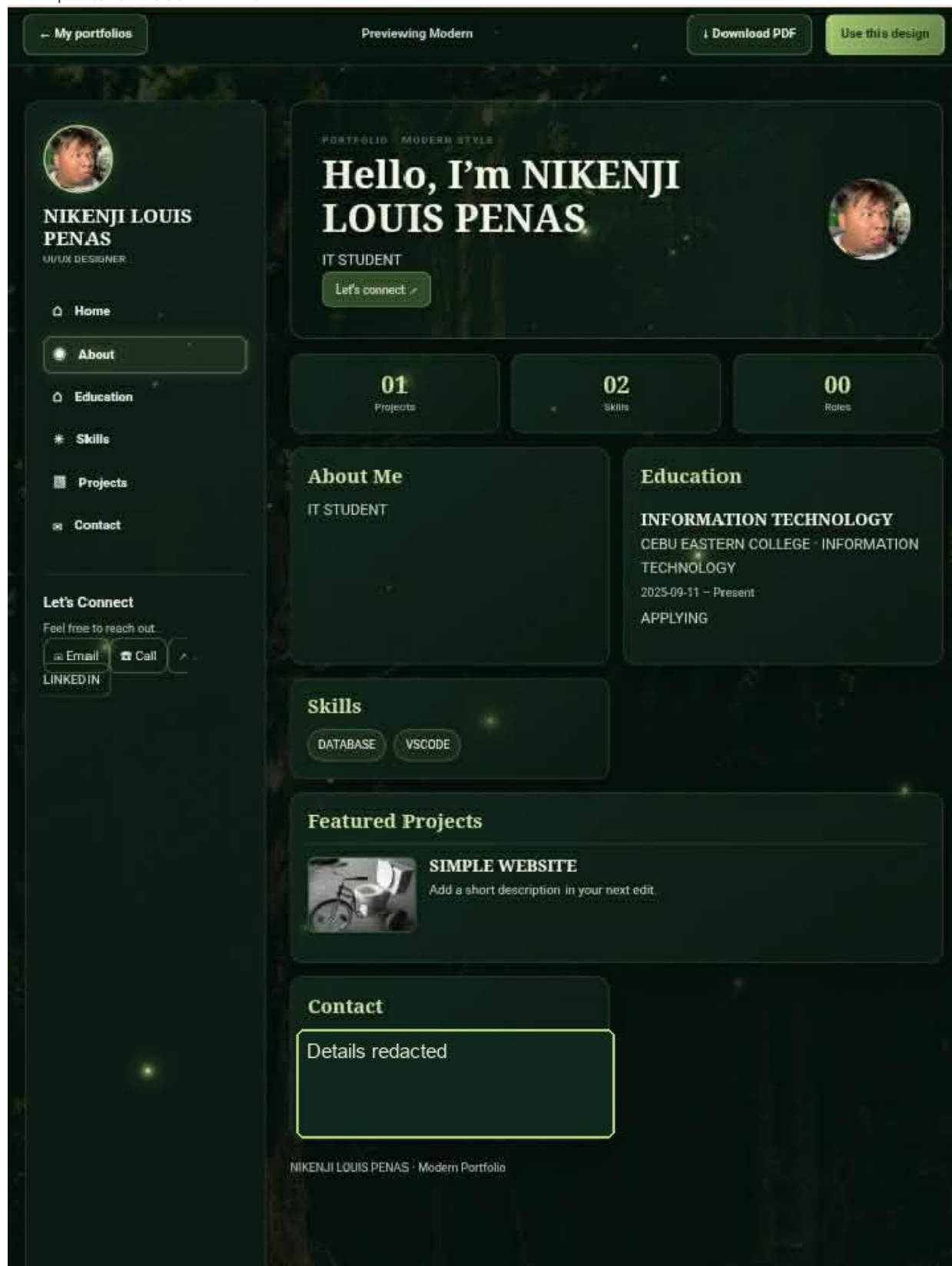


Figure 7. Existing Modern preview screenshot. Personal contact values are redacted in this documentation copy.

Creative

A framed, more expressive arrangement with a warm border treatment and an asymmetric content grid. It renders the same portfolio data through its own CreativeTemplate component.



Figure 8. Existing Creative preview screenshot. Personal contact values are redacted in this documentation copy.

Template selection and preview are implemented separately: the chooser links to an owner preview route and submits the selected key through a server action. The current stored key is constrained to the three supported values in the migration.

G. Online Database and Database Structure

Platform: the source contains Supabase SQL migrations that target PostgreSQL, plus Supabase client calls. This is evidence of the intended/configured application data model, not direct confirmation of the live Supabase project’s schema, policies, bucket state, or data.

Table	Columns from migration definitions (name: type; key/default notes)
public.portfolios	id: uuid PK default gen_random_uuid(); user_id:

Table	Columns from migration definitions (name: type; key/default notes)
	uuid NOT NULL FK auth.users(id) ON DELETE CASCADE; slug: text NOT NULL UNIQUE; full_name: text NOT NULL; email: text NOT NULL; role: text NOT NULL default ""; about_me: text NOT NULL default ""; template_key: text NOT NULL default 'modern' CHECK modern/creative/minimal; profile_photo_path: text nullable; skills: jsonb NOT NULL default []; projects: jsonb NOT NULL default []; is_published: boolean NOT NULL default false; created_at, updated_at: timestamptz NOT NULL default now(); contact_number, address: text nullable; deleted_at: timestamptz nullable.
public.portfolio_education	id: uuid PK default gen_random_uuid(); portfolio_id: uuid NOT NULL FK portfolios(id) ON DELETE CASCADE; school: text NOT NULL; degree, field_of_study: text nullable; start_date, end_date: date nullable; currently_studying: boolean NOT NULL default false; description: text nullable; sort_order: integer NOT NULL default 0; created_at: timestamptz NOT NULL default now().
public.portfolio_experiences	id: uuid PK default gen_random_uuid(); portfolio_id: uuid NOT NULL FK portfolios(id) ON DELETE CASCADE; position: text NOT NULL; company: text nullable; start_date, end_date: date nullable; currently_working: boolean NOT NULL default false; description: text nullable; sort_order: integer NOT NULL default 0; created_at: timestamptz NOT NULL default now().
public.portfolio_social_links	id: uuid PK default gen_random_uuid(); portfolio_id: uuid NOT NULL FK portfolios(id) ON DELETE CASCADE; platform, url: text NOT NULL; sort_order: integer NOT NULL default 0; created_at: timestamptz NOT NULL default now().

Relationships and indexes: one Supabase Auth user can own multiple portfolios; each portfolio can have multiple education, experience, and social-link rows. Child records cascade when their parent portfolio is removed. The three child tables have

portfolio/order indexes. Skills and projects are JSONB arrays on the parent table, not separate relational tables.

Row-level security in migration files: portfolio owners can select/insert/update/delete their own rows.

Anonymous/authenticated visitors can select only published portfolios whose deleted_at is null.

Related education, experience, and social-link records have owner-scoped operations and public reads only while their parent is published and active. This description is from SQL source; applying and verifying the policies in the hosted database remains unconfirmed.

Storage: the migrations define a bucket named portfolio-media and permit PNG, JPEG, and WebP. A later migration makes the bucket private and sets a 50 MB per-file limit. Upload paths begin with the authenticated user's UUID. Storage policies allow owners to manage their own files and allow reads for owners or assets referenced by an active published portfolio.

Server code creates signed image URLs valid for 600 seconds.

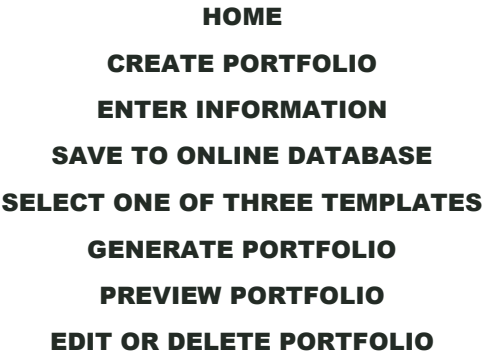
CRUD path in code: create inserts a portfolio then upserts child rows; read queries filter by owner or published slug; edit updates the portfolio and synchronizes child rows; deletion first sets deleted_at and turns publication off; recovery can restore the record; permanent deletion removes the portfolio and then attempts to remove its associated image objects.

These flows were inspected in source but were not exercised against the live database for this report.

Live configuration boundary: no Supabase dashboard inspection or query was performed. Actual rows, applied migration versions, Auth redirect allow-list, Storage bucket settings, and policies are therefore unverified.

H. System Flow

The diagram shows the intended user flow represented by the routes and action code. It does not imply that every external integration was tested.



The code also includes publication controls: an owner can make a saved portfolio public, after which its /p/{slug} page is queryable; deleting it moves it to Recovery and removes it from public results.

I. Deployment Information

Item	Documented value and verification boundary
------	--

Item	Documented value and verification boundary
Live website	https://everleaf-portfolio-generator.vercel.app/home (URL supplied in the brief; live accessibility could not be verified with the available page-fetch tool).
GitHub	NikenjiPenas/Everleaf-portfolio-generator/tree/nextjs-vercel; the inspected branch was nextjs-vercel at commit 27a06a3.
Hosting	Vercel, as supplied by the project owner and repository overview. Vercel dashboard Root Directory, build settings, environment values, and latest deployment status were not inspected.
Application root	next-app/ contains package.json, next.config.ts, and the App Router source. Vercel Root Directory should be set to next-app; this dashboard setting itself remains unverified.
Database	Supabase-hosted PostgreSQL is the platform represented by the checked-in migrations and application clients. Live project linkage was not independently verified.
Connection	The app reads NEXT_PUBLIC_SUPABASE_URL and NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY. NEXT_PUBLIC_SITE_URL is used to construct confirmation and recovery callbacks; NEXT_PUBLIC_SUPABASE_MEDIA_BUCKET defaults to portfolio-media.

The expected deployment environment variables are documented as names and placeholders in the root README and

next-app/env.example. The contents of the local .env.local file were not read or copied. Production Auth settings should

allow the production origin and /auth/callback redirect; the dashboard configuration requires owner verification.

Local build check: PASS. npm run build completed for Next.js 16.3.8 and TypeScript. The first run inside the restricted

sandbox failed while SWC canonicalized the project path (Access is denied); rerunning with filesystem access allowed

completed successfully. This build result is local and does not prove the remote Vercel build is current.

J. Testing and Results

The Next.js production build is the only runtime-oriented check performed during this documentation task. The interaction

tests below were not run, so none are marked **PASS**. Existing screenshots and source code are not substitutes for

end-to-end test evidence.

Test case	Expected result	Actual result	Status
Open published website	Home page loads at the published URL.	Could not verify live accessibility from this environment.	NOT TESTED
Register or log in	User account/session is created or accepted.	Auth flow exists in source; not exercised.	NOT TESTED
Enter portfolio information	Valid profile fields can be entered.	Form route exists; not exercised.	NOT TESTED
Save portfolio information	Parent and detail records are saved.	Server action inspected; no live save performed.	NOT TESTED
Verify saved database record	Saved values appear in Supabase.	Supabase project was not queried.	NOT TESTED
Retrieve an existing portfolio	Owner can reopen their saved record.	Read paths exist; not exercised.	NOT TESTED
Select Simple template	Simple design is selected and saved.	minimal key/component exists; not exercised.	NOT TESTED
Select Modern template	Modern design is selected and saved.	modern key/component exists; not exercised.	NOT TESTED
Select Creative template	Creative design is selected and saved.	creative key/component exists; not exercised.	NOT TESTED
Generate and preview	Preview renders saved portfolio data.	Preview route exists; not exercised.	NOT TESTED
Edit existing portfolio	Owner can update portfolio data.	Update action exists; not exercised.	NOT TESTED
Verify updated information	Saved changes are reflected after reload.	No edit/reload test performed.	NOT TESTED
Delete a portfolio	Portfolio moves to Recovery and is unpublished.	Soft-delete action exists; not exercised.	NOT TESTED
Verify deleted record unavailable	Deleted portfolio is not public or active.	SQL and query filters inspected; not exercised.	NOT TESTED
Refresh and verify persistence	Saved record remains available after refresh.	No live persistence test performed.	NOT TESTED

Test case	Expected result	Actual result	Status
Test mobile responsiveness	Pages remain usable at mobile widths.	Responsive CSS exists; no device/browser test performed.	NOT TESTED

Build evidence: npm run build succeeded and printed the compiled App Router route list. The package also declares npm

run check (TypeScript no-emit). No claim of a successful CRUD test, Supabase verification, mobile visual test, or live

deployment test is made here.

K. Limitations and Recommendations

Verified limitations and open checks

Finding	Practical next step
Hosted Supabase schema and RLS state are not verified.	Apply migrations in order to the intended project, then inspect tables, policies, and bucket settings in Supabase.
Vercel dashboard settings are not verified.	Confirm Root Directory is next-app, build command is npm run build, and all required variables are set for Production/Preview as intended.
Live website fetch was unavailable here.	Open the provided Vercel URL in a browser and capture a fresh, dated screenshot after confirming current deployment.
Interaction tests were not run.	Use the test matrix in Section J with a dedicated demo account and safe sample data; record actual results and evidence.
No dedicated Next.js test runner is declared in next-app/package.json.	Add an automated test suite only if it is within project scope; otherwise record a manual acceptance test session.
Existing screenshots may reflect earlier page versions and sample profiles.	Replace them with current, consented demo captures before final academic submission; keep contact details and private account data hidden.
The global stylesheet uses a remote Unsplash background image URL.	Check production availability and licensing; consider a locally managed asset if stable background rendering is required.
The submitted Supabase email redirect settings are external to this repository.	Verify local and production Site URL/redirect allow-list settings without exposing credentials.

The root includes Laravel application files and a Render configuration retained from an earlier implementation. The

checked-in README identifies next-app as the current Vercel app; only the live Vercel project settings can confirm which source directory is actually deployed.

L. Conclusion

EverLeaf's current repository contains a Next.js portfolio generator with three distinct template components,

Supabase-backed authentication/data/storage code, portfolio management and recovery flows, and public portfolio

rendering. The checked-in migrations describe an owner-scoped PostgreSQL model with published-read policies, while

screenshots document several existing UI views.

The local production build passed. The hosted database configuration, live deployment settings, public-site accessibility,

and manual interaction/mobile tests remain unverified. The project should be described as implemented in source and

buildable, with the listed external checks and acceptance tests still to be completed.

Appendix: screenshots

Available repository screenshots are stored in documentation/screenshots/. The checklist identifies what is present, what

was cropped from an existing capture, and which database/edit/delete/mobile screenshots still need a real manual capture.

Contact values were redacted in the template screenshots embedded in this report.